NGINX
Beginner Study Notes

A practical guide to web servers, HTTP requests, configuration, Linux services, logs, reverse proxies and basic troubleshooting.

Learning goal: Understand what NGINX does, how a browser request reaches it, where its Ubuntu files live, which commands matter, and what the key terms mean. This guide covers beginner-level knowledge—not advanced production administration.

Designed for learning alongside Ubuntu / WSL and basic Linux administration.

1. What is NGINX?

NGINX (pronounced “engine-x”) is web-server software. A web server receives requests from clients—usually browsers—and sends responses. NGINX can serve website files directly and can sit in front of application servers.

Example: When you open `http://localhost`, your browser sends an HTTP request to your computer. If NGINX is running and listening on the requested port, it can return a webpage.

Why is NGINX used?

- **Web server:** serves HTML, CSS, JavaScript, images and other static files.
- **Reverse proxy:** receives requests and forwards them to an application server.
- **Load balancer:** distributes requests across multiple backend servers.
- **Caching:** can store eligible responses to reduce repeated work.
- **TLS/HTTPS termination:** can handle HTTPS encryption at the front of an application stack.

2. How a web request works

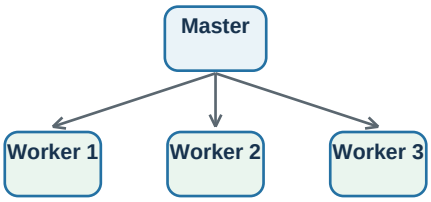
Step	What happens
1. Request	A browser asks for a page, such as GET /.
2. Listener	The request reaches the IP address and port where NGINX is listening.
3. Configuration	NGINX selects a matching server block and location rule.
4. Processing	NGINX reads a file, returns a response, or proxies the request to an application.
5. Response	NGINX sends an HTTP response to the client.

Common HTTP status codes

Code	Meaning
200 OK	Request succeeded.
301 / 302	Redirect to another URL.
403 Forbidden	Access is refused.
404 Not Found	Requested resource was not found.
500 Internal Server Error	A server-side error occurred.
502 Bad Gateway	A proxy received an invalid response from an upstream server.
504 Gateway Timeout	A proxy did not receive a timely response from an upstream server.

3. NGINX process architecture

NGINX commonly uses a **master process** and one or more **worker processes**. The master manages configuration and workers; workers handle client connections and requests.



- **Master process:** reads configuration, starts/manages workers and handles reload tasks.
- **Worker processes:** handle client connections and serve or proxy requests.

The number of workers depends on configuration and environment. You do not need to tune it as a beginner.

4. Common ports and protocols

Protocol / port	What to remember
HTTP — 80	Common default port for unencrypted HTTP.
HTTPS — 443	Common default port for HTTP protected by TLS.
SFTP — 22	Secure file transfer over SSH; not an NGINX web port.

These are defaults, not guarantees. Administrators can configure different ports; firewalls can also affect reachability.

5. Important NGINX locations on Ubuntu

Path	Typical purpose
/etc/nginx/	Main NGINX configuration directory.
/etc/nginx/nginx.conf	Main configuration file.
/etc/nginx/sites-available/	Ubuntu/Debian convention for available site configurations.
/etc/nginx/sites-enabled/	Ubuntu/Debian convention for enabled sites, often symbolic links.
/var/www/html/	Common default website directory on Ubuntu.
/var/log/nginx/access.log	Records requests received by NGINX.
/var/log/nginx/error.log	Records warnings and errors reported by NGINX.

These are common Ubuntu package defaults. Custom installations may use different paths.

```
ls /etc/nginx
ls /var/www/html
ls /var/log/nginx
sudo cat /etc/nginx/nginx.conf
```

6. Basic NGINX configuration

NGINX configuration contains **directives** grouped into blocks. A directive usually ends with a semicolon; curly braces group related settings.

```
server {
    listen 80;
    server_name localhost;

    location / {
        root /var/www/html;
        index index.html;
    }
}
```

Part	Meaning
server { ... }	Defines a virtual server configuration block.
listen 80;	Listen for HTTP requests on port 80.
server_name localhost;	Names this server block for matching requests.
location / { ... }	Defines handling for requests whose path begins with /.
root /var/www/html;	Sets the filesystem root used to construct a file path.
index index.html;	Names the default index file for directory requests.

This is a simplified teaching example. The active Ubuntu default site may contain more directives and may be stored in a separate site file.

Test configuration before applying changes

```
sudo nginx -t
```

If valid, NGINX reports that the test is successful. If not, read the error message; it often identifies the file and line number.

Apply configuration changes

```
sudo systemctl reload nginx
```

A reload asks a running NGINX service to load updated configuration without a full stop/start. Test first; do not reload a configuration that fails validation.

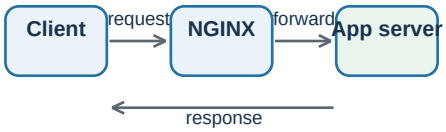
7. Essential service commands

Command	Purpose
nginx -v	Display NGINX version.
systemctl status nginx	Show service status.
sudo systemctl start nginx	Start NGINX now.
sudo systemctl stop nginx	Stop NGINX now.
sudo systemctl restart nginx	Stop and start the service again.
sudo systemctl reload nginx	Reload configuration.
sudo systemctl enable nginx	Configure automatic start at system boot.
sudo systemctl disable nginx	Disable automatic start at system boot.
sudo nginx -t	Test configuration.

Start means run now. **Enable** means start automatically at boot. They are different operations.

8. Reverse proxy

A reverse proxy sits between clients and backend applications. The client connects to NGINX; NGINX forwards the request to an application and returns the response.



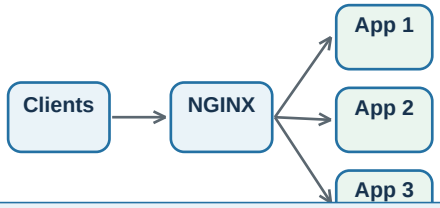
Example: a Python application listens on port 5000. NGINX can listen on port 80 and forward matching requests to the Python application.

```
location / {
    proxy_pass http://127.0.0.1:5000;
}
```

This is a minimal illustration. Real deployments may require proxy headers, timeouts, TLS and application-specific settings.

9. Load balancing

A load balancer distributes incoming requests across multiple backend servers. This can spread work and improve resilience, depending on the architecture.



Remember the concept: NGINX can distribute traffic among several application servers.

10. Logs

Log	What it tells you
access.log	Requests received by NGINX; useful for checking whether a client reached it.
error.log	Problems NGINX reports, such as configuration, permission, file or upstream errors.

```
# Show recent access log entries
sudo tail -n 20 /var/log/nginx/access.log

# Follow access log live; Ctrl+C to stop
sudo tail -f /var/log/nginx/access.log

# Follow error log live; Ctrl+C to stop
sudo tail -f /var/log/nginx/error.log
```

An access-log entry generally shows that NGINX received a request; it does not prove that the entire application workflow succeeded.

11. Hands-on practice in Ubuntu / WSL

If NGINX is already installed in your WSL Ubuntu, try these safe checks. You may need sudo for service management and protected files.

A. Check installation and version

```
nginx -v
```

B. Check service status

```
systemctl status nginx
```

Look for `Active: active (running)`. Press q to exit the status pager.

C. Test configuration

```
sudo nginx -t
```

D. Request the local page

```
curl -I http://localhost
```

The `-I` option requests response headers. To see the response body, run `curl http://localhost`.

E. Check website files

```
ls -l /var/www/html
```

F. See what is listening on port 80 (if available)

```
sudo ss -ltnp | grep ':80'
```

If a command fails, note the exact error. NGINX may be uninstalled, stopped, listening on another port, or systemd may not be active in WSL.

12. Basic troubleshooting

Symptom	First checks
Connection refused	Check service status and whether NGINX listens on the expected port.
Configuration test fails	Run sudo nginx -t and inspect the reported file and line.
404 Not Found	Check URL, matching location, document root and file name.
403 Forbidden	Check file/directory permissions and NGINX access rules. Do not make everything world-writable.
502 Bad Gateway	For a reverse proxy, check the backend is running and proxy_pass address/port is correct.
Changes do not appear	Test configuration, reload NGINX, and check browser cache and served file.

13. NGINX vs Apache (very basic)

NGINX	Apache HTTP Server
Known for an event-driven architecture and efficient handling of many concurrent connections.	Offers several process/thread models and a large module ecosystem.
Often used as a reverse proxy and load balancer.	Often used as a web server and also supports proxy features.
Configuration commonly uses nginx.conf and included files.	Configuration layout varies by OS and installation.

Both are capable web-server platforms. The right choice depends on application and operational requirements.

14. Security habits

• Prefer HTTPS for real websites, especially when handling sensitive information.
• Keep NGINX and the operating system updated.
• Use least-privilege permissions; avoid `chmod 777` as a shortcut.
• Do not expose backend ports or admin interfaces unnecessarily.
• Protect private keys and credentials; do not put secrets in public web directories.
• Test configuration before reloading and inspect logs after changes.

15. WSL-specific notes

• WSL Ubuntu runs alongside Windows; it is not automatically a separate internet-facing server.
• In common WSL networking setups, Windows can reach a service in WSL through `http://localhost`, if the service is running and listening appropriately.
• Service management depends on WSL's systemd/service setup. If systemctl says the system was not booted with systemd, that is a WSL setup issue—not necessarily an NGINX issue.
• For local Windows-to-WSL file access, FileZilla is usually unnecessary; it is mainly useful for a separate remote server.

16. Quick revision sheet

Question	Remember
What is NGINX?	Web-server software; can also be a reverse proxy and load balancer.
What does a web server do?	Receives client requests and sends HTTP responses.
Master process?	Manages configuration and worker processes.
Worker processes?	Handle client connections and requests.
HTTP / HTTPS default ports?	80 / 443.
Main Ubuntu config?	Usually /etc/nginx/nginx.conf.
Common Ubuntu web root?	Often /var/www/html.
Logs?	Often /var/log/nginx/access.log and error.log.
Test config?	sudo nginx -t
Reload service?	sudo systemctl reload nginx
Check status?	systemctl status nginx

17. Practice checklist

■ Explain NGINX in your own words.
■ Draw Browser → NGINX → webpage.
■ Explain master vs worker process.

- Find the configuration, web-root and log directories.
- Run nginx -v, systemctl status nginx and sudo nginx -t.
- Explain start, stop, restart, reload and enable.
- Explain reverse proxy and load balancing at a high level.

Learning boundary: For a beginner learning Linux, these notes cover the NGINX fundamentals you need now. You do not need advanced TLS, caching rules, complex load-balancing algorithms or production tuning yet.

End of beginner study notes